Informe técnico | Reto R010 Banistmo

Automatización E2E · Java 17 · Gradle · Serenity BDD · Screenplay · Cucumber

Resultado de ejecución · PASSED · 1 escenario · 9 pasos · 18 s · Chrome / Windows

1. Objetivo y cumplimiento del reto

El objetivo fue automatizar el flujo solicitado de Asistencia Pyme hasta la consulta de sucursales y dejar una solución reproducible, versionada y con evidencia. El escenario navega desde Personas → Pymes → Productos y servicios → Asistencia pyme → "Solicítalo con tu Ejecutivo Pyme", filtra Tipo de punto = "Sucursales" y Provincia = "Colón", valida exactamente 2 sucursales y confirma "Zona Libre" asociada a Colón.

Requisitos cubiertos

- Java 17 + Gradle para build y ejecución.
- Patrón Screenplay con Tasks, Interactions, Questions, Targets, Step Definitions y Runner.
- Serenity BDD + Selenium WebDriver para automatización y reporte.
- Cucumber/Gherkin para expresar el comportamiento en lenguaje de negocio.
- Repositorio Git/GitHub con evolución incremental y comandos documentados.
- Reporte de resultados con estado, duración, pasos y capturas.
- La versión entregada debe conservarse sin cambios posteriores para mantener trazabilidad.

2. Escenario automatizado

```
Característica: Asistencia Pyme y búsqueda de sucursales
Escenario: Consultar Asistencia Pyme y validar una sucursal en Colón
  Dado que el usuario ingresa al portal de Personas de Banistmo
  Cuando navega a la sección Pymes
  Y selecciona Productos y servicios
  Y ingresa a Asistencia pyme
  Y selecciona Solicítalo con tu Ejecutivo Pyme
  Entonces debe visualizar el buscador emergente de sucursales correcto
  Cuando busca el tipo de punto "Sucursales" en la provincia "Colón"
  Entonces debe visualizar 2 sucursales en la provincia "Colón"
  Y debe visualizar la sucursal "Zona Libre" asociada a "Colón"
```

3. Arquitectura y decisiones

Flujo: Feature/Gherkin → Step Definitions → Tasks/Questions → Interactions → Targets/UI → Serenity + Selenium → Portal Banistmo.

Screenplay separa intención de negocio y detalle técnico; Serenity aporta trazabilidad y evidencia; Java 17 ofrece una base LTS estable; Gradle centraliza dependencias, ejecución y generación de reportes. Los datos funcionales variables permanecen visibles en Gherkin para evitar valores ocultos en el código.

4. Evolución del código por commits

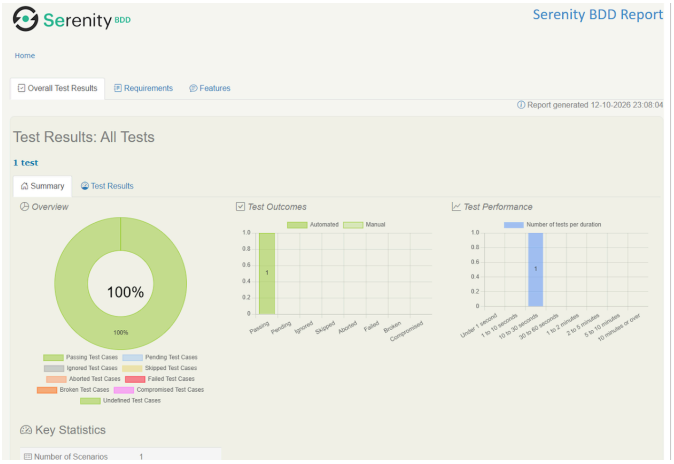
01 bc9080c - Inicio del proyecto con Screenplay: línea base, estructura y flujo inicial.
02 6b1764f - Manejo del aviso de privacidad y ventanas emergentes: resiliencia ante bloqueos opcionales.
03 43bd26b - Ajuste del acceso al buscador de sucursales: robustecimiento del CTA y transición.
04 f99cafa - Selección de filtros con scroll automático: interacción estable con controles dinámicos.
05 fa7a33e - Validación de las sucursales disponibles en Colón: conteo exacto y validación funcional.
06 b1f603c - Optimización de tiempos de espera y capturas: reducción de esperas y lectura más eficiente del DOM.
07 c6fe222 - Ajuste de la selección de provincia: sincronización adaptativa del segundo filtro.
08 c445504 - Mejora de evidencias en el reporte de Serenity: estabilización del viewport para las capturas.
Cierre · Ajustes finales de calidad y documentación — consolidación del README, informe técnico y verificación final.

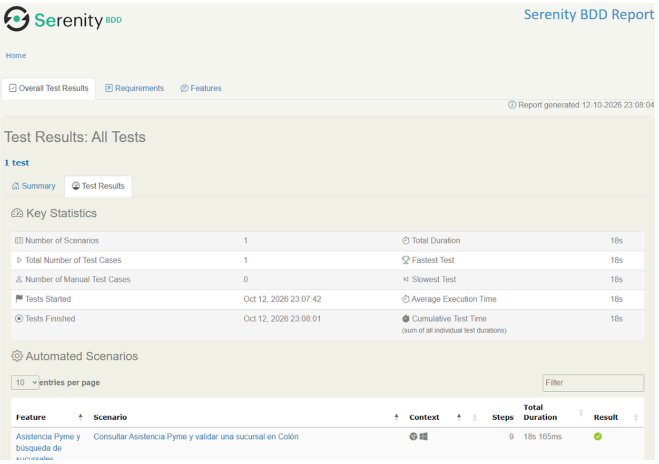
5. Control de versiones y comandos Git

`Revisión: git status | git diff --name-status | git diff --stat`
`Staging controlado: git add -A -- <ruta> | git diff --cached --name-status | git diff --cached --stat`
`Commit: git commit -m "<mensaje descriptivo>"`
`Publicación: git push origin main`
`Historia: git log --oneline`
El staging se realizó por rutas específicas para evitar incluir target/, build/, wrappers temporales u otros artefactos no relacionados.

6. Resultado y evidencia

Serenity registró 1 escenario y 1 caso de prueba con resultado PASSED, 9 pasos y duración total de 18 s. Las etiquetas visibles son E2E, R010, Banistmo y Screenplay; el contexto de la evidencia es Chrome / Windows.





La vista general confirma 100 % de éxito y la vista Test Results muestra el escenario "Consultar Asistencia Pyme y validar una sucursal en Colón" con estado satisfactorio. El detalle de pasos y capturas permanece en target/site/serenity/index.html.

7. Ejecución y reproducción

`Windows: .\gradlew.bat clean test aggregate`
`Linux/macOS: ./gradlew clean test aggregate`
`Headless: ./gradlew clean test aggregate -Dheadless.mode=true`
`Reporte local: target/site/serenity/index.html`
`GitHub CI: Actions → Banistmo R010 Automation CI → Run workflow → main → job e2e → Artifacts → serenity-report.`
El workflow configura Java 17, ejecuta en modo headless y publica el reporte Serenity como artefacto.

8. Conclusión

La solución cumple el escenario solicitado y los requisitos técnicos del reto: Java, Gradle, Screenplay, Serenity/Selenium, Cucumber/Gherkin, Git, comandos documentados y reporte de resultados. La separación por commits permite revisar la evolución desde la línea base hasta una automatización más resiliente, rápida y auditable. Una vez entregado el repositorio evaluable, su estado debe conservarse para mantener correspondencia entre código, historial Git y evidencias.